Cahier des charges — TGFT

Plateforme communautaire modulaire pour groupes de joueurs Teamfight Tactics

Projet de fin de bachelier — deuxième version Cédric Van Overlop

Sommaire

1. Introduction
2. User stories
3. La marketplace de modules
4. Stack technique et justifications
5. Défis techniques et points d'intérêt

1. Introduction

1.1 Le domaine, pour situer

Teamfight Tactics (TFT, Riot Games) est un jeu de stratégie en ligne du genre *auto-battler*. Huit joueurs s'affrontent dans la même partie, chacun sur son propre plateau hexagonal.

À chaque tour, un joueur achète des personnages dans une boutique aléatoire et les dispose sur sa grille. Puis vient le combat contre un adversaire — et **le joueur ne contrôle plus rien** : l'affrontement se résout automatiquement. Toute la compétence s'exerce donc *avant* le combat. C'est un jeu de préparation, plus proche des échecs que d'un jeu d'action.

Trois savoir-faire distincts composent cette préparation :

Compétence	En quoi ça consiste
Composition	Quels personnages assembler pour activer des synergies d'équipe
Économie	Quand dépenser son or, quand épargner
Positionnement	Où placer précisément chaque unité sur les cases du plateau

1.2 Le problème

Les communautés de joueurs organisées — serveurs Discord thématiques, associations proposant du coaching, groupes d'entraînement restreints — partagent toutes le même besoin : progresser collectivement, en mutualisant les connaissances et en discutant des situations de jeu.

Elles n'ont pour cela aucun outil adapté. En pratique, elles improvisent :

- des **captures d'écran** postées dans un salon Discord pour discuter d'un placement, impossibles à modifier ou à reprendre ;
- des **tableurs partagés** pour lister les compositions viables du moment, jamais à jour ;
- des **explications en vocal**, qui disparaissent dès qu'elles sont données et qu'il faut répéter à chaque nouveau membre.

Le résultat est une **perte d'information permanente**. Ce qu'un joueur expérimenté explique un soir n'existe plus le lendemain, et aucune trace consultable ne se constitue.

1.3 La proposition

TGFT est une plateforme web destinée aux communautés TFT organisées, permettant à chaque groupe de construire et partager sa propre base de connaissance et ses propres outils de travail.

Elle repose sur quatre fonctionnalités de base, dont **deux sont engagées** au titre du MVP et deux différées (voir §2) :

- **Team Builder** *(engagé)* — un éditeur de plateau où l'on construit une situation de jeu, avec calcul automatique des synergies, et dont l'état complet est encodé dans l'URL : le lien devient l'objet partageable, sans rien stocker.
- **Tier Lists** *(engagé)* — le classement collaboratif de ce qui est fort en ce moment, sur cinq catégories (compositions, augments, objets composables, emblèmes, objets forgés), réordonnable par les contributeurs du groupe.
- **Compos** *(différé)* — des fiches de composition documentées et maintenues par les membres expérimentés : plateau de référence, objets recommandés, conditions de viabilité, conseils de jeu.
- **Spotchecks** *(différé)* — des situations de choix soumises au débat du groupe, pour travailler la prise de décision.

1.4 L'ambition réelle du projet : une base open source modulaire

L'intérêt du projet ne réside pas dans ces fonctionnalités, mais dans l'architecture qui les porte.

Les communautés visées n'ont pas les mêmes besoins. Un serveur public de plusieurs milliers de membres cherche du contenu accessible ; un groupe d'entraînement de quinze personnes cherche du suivi sérieux et de la confidentialité ; une association organisant des tournois cherche des outils de compétition. Une application unique qui contenterait les trois serait illisible pour tout le monde.

TGFT est donc conçue comme **une base open source extensible** :

- un **noyau minimal** — comptes, groupes, membres, rôles, permissions ;
- des **modules** portant les fonctionnalités, que chaque groupe active, désactive et configure selon ses besoins ;
- une **marketplace ouverte** : n'importe quel développeur peut écrire un module et le publier lui-même, **sans relecture du mainteneur ni redéploiement de la plateforme**, son code s'exécutant dans un environnement isolé (voir §3).

Le modèle de référence est celui d'Odoo pour l'activation des modules par groupe, et celui des Shopify Functions pour l'exécution du code écrit par des tiers.

Le projet distingue donc **deux catégories de modules** :

	Modules natifs	**Modules externes**
Exemples	Team Builder, Tier Lists	Tournoi (module de démonstration)
Auteur	Le mainteneur du projet	N'importe quel développeur
Technologie	C# et Angular, dans le code de la plateforme	JavaScript compilé en WebAssembly, dans le dépôt du développeur
Publication	Avec une nouvelle version de la plateforme	À tout moment, sans relecture ni redéploiement
Confiance	Totale : code relu, testé et compilé avec le noyau	Aucune : le module est traité comme potentiellement hostile

Contrainte de conception fondatrice : les deux catégories partagent les mêmes contrats — un manifeste qui déclare le module, ses écrans, ses réglages et ses commandes Discord ; l'activation par groupe ; les permissions par

rôle ; l'obligation de purge et d'anonymisation. Seule l'exécution diffère : les modules natifs tournent avec la plateforme, les modules externes dans un bac à sable.

> **Pourquoi une marketplace alors qu'aucun contributeur externe n'est attendu pendant le projet ?** Parce que sa valeur première n'est pas l'ouverture, mais la **contrainte qu'elle impose au développeur lui-même**. Un système où toutes les fonctionnalités passent par des contrats publics ne peut pas dériver vers le monolithe accidentel : chaque raccourci est un franchissement de frontière, et les tests d'architecture le refusent. La marketplace est démontrée de bout en bout par un **module de démonstration Tournoi**, écrit dans un dépôt séparé exactement comme le ferait un développeur inconnu. L'ouverture à des tiers est une conséquence de cette discipline, pas sa justification.

1.5 Modèle d'exploitation

Instance unique, hébergée par l'auteur. Les utilisateurs s'inscrivent et rejoignent les groupes de leur choix ; chaque groupe dispose de ses propres données, strictement isolées de celles des autres. C'est une architecture **multi-tenant**.

Un utilisateur peut appartenir à plusieurs groupes, avec un rôle différent dans chacun.

Le code des modules externes s'exécute lui aussi sur cette instance : **un développeur n'a aucun serveur à héberger**. L'ensemble tient dans un **budget d'hébergement inférieur à 10 € par mois**, sur un serveur virtuel unique — contrainte qui exclut toute architecture fondée sur un conteneur ou une base de données par groupe ou par module.

1.6 Prolongement Discord

Ces communautés vivant déjà sur Discord, la plateforme s'y prolonge par un **bot** : consultation des tier lists et des plateaux directement depuis un salon, publication automatique des fiches de composition. Le bot n'est pas une fonctionnalité prioritaire, mais il conditionne l'adoption réelle de l'outil.

1.7 Cadre légal

Licence du projet : AGPL-3.0, avec régime de contribution DCO pour le dépôt principal.

L'argument déterminant — la clause réseau, propre à l'AGPL. Elle étend l'obligation de publication au cas où un logiciel modifié est **mis à disposition via un réseau**, sans être redistribué. Elle vise un tiers qui reprendrait le projet pour en exploiter une version fermée en tant que service. Une licence GPL classique ne couvrirait pas ce cas, une licence permissive encore moins. Le projet étant lui-même destiné à être exploité comme service en ligne, c'est le risque principal.

Les modules natifs. Compilés dans le même binaire que le noyau, ils constituent une œuvre liée et relèvent donc de la même licence : c'est le copyleft par liaison de code, commun à la GPL et à l'AGPL.

Les modules externes. Ils vivent dans le dépôt de leur auteur et ne communiquent avec la plateforme que par échange de données. Ce lien ne permet pas de les considérer avec certitude comme des œuvres liées : la licence du noyau ne s'impose donc pas à eux. Pour que le catalogue reste libre, **une règle de publication prend le relais** : un module n'est accepté que si son code source est public, sous une licence libre reconnue par l'OSI. C'est aussi cette condition qui rend possible la compilation attestée depuis le dépôt (§3.5).

Cadre Riot Games : usage strictement non commercial, mention « non approuvé par Riot » visible, aucune interaction avec le client de jeu, et **aucun système reproduisant ou estimant le classement officiel** — explicitement interdit par leurs conditions d'utilisation. Ces règles s'imposent aussi aux modules externes et figurent dans les conditions de publication de la marketplace. Les données de jeu (unités, objets, synergies, augments, illustrations) proviennent de **Data Dragon**, le dépôt de données statiques officiel de Riot Games. Il s'agit d'un contenu public ne requérant **aucune clé d'API**, ce qui supprime toute dépendance à une autorisation d'accès et tout risque de révocation d'identifiant applicatif. La version publiée est vérifiée une fois par jour, et les données ne sont rechargées qu'en cas de changement effectif.

2. User stories

2.1 Niveaux de priorité

Niveau	Signification
MVP	Sans cette fonctionnalité, le produit n'existe pas. Engagement ferme
P2	Attendu, développé si le MVP est tenu
P3	Bonus, hors engagement

Aucune fonctionnalité du bot Discord n'est classée MVP : le bot est développé indépendamment de l'application web, une fois le socle web stabilisé.

2.2 Application web

Épique A — Compte et accès

Réf.	User story	Prio
A1	En tant que visiteur, je veux me connecter avec mon compte Discord, afin d'accéder à la plateforme sans créer de nouveaux identifiants	**MVP**
A2	En tant qu'utilisateur connecté, je veux que mon pseudo et mon avatar Discord soient repris automatiquement, afin d'être reconnaissable par les membres de mon groupe	**MVP**
A3	En tant qu'utilisateur, je veux consulter et exporter les données que la plateforme détient sur moi, afin d'exercer mes droits d'accès et de portabilité	**MVP**
A4	En tant qu'utilisateur, je veux supprimer définitivement mon compte, afin d'exercer mon droit à l'effacement	**MVP**
A5	En tant qu'utilisateur supprimant son compte, je veux que mes contributions au groupe restent en place mais anonymisées, afin de ne pas détruire le travail collectif	**MVP**
A6	En tant que visiteur, je veux consulter la politique de confidentialité et les conditions d'utilisation, afin de savoir à quoi je consens avant de m'inscrire	**MVP**
A7	En tant qu'utilisateur, je veux rester connecté entre mes visites, afin de ne pas me réauthentifier à chaque fois	P2

> **A3 à A6 sont classées MVP pour raison légale, non fonctionnelle.** L'exploitation d'une instance unique hébergeant les données de plusieurs communautés fait de l'auteur le **responsable de traitement** au sens du RGPD. Les droits d'accès, de portabilité et à l'effacement, ainsi que l'information préalable des utilisateurs, ne sont pas des améliorations : sans eux, la plateforme ne peut pas être exploitée légalement. Le confort de session (A7), lui, n'a aucune incidence de ce type.

Épique B — Groupes

Réf.	User story	Prio
B1	En tant qu'utilisateur, je veux créer un groupe, afin de rassembler ma communauté sur la plateforme	**MVP**
B2	En tant que fondateur, je veux définir la visibilité de mon groupe (public ouvert, public sur candidature, privé sur invitation), afin de contrôler qui peut le rejoindre	**MVP**
B3	En tant qu'utilisateur, je veux appartenir à plusieurs groupes et basculer facilement de l'un à l'autre, afin de participer à plusieurs communautés	**MVP**

Réf.	User story	Prio
B4	En tant qu'administrateur, je veux générer un lien d'invitation, afin de faire entrer des membres sans les ajouter un par un	**MVP**
B5	En tant qu'utilisateur, je veux parcourir les groupes publics et demander à en rejoindre un, afin de trouver une communauté	**MVP**
B6	En tant que fondateur, je veux personnaliser l'identité visuelle de mon groupe (nom, logo, couleur), afin qu'il soit reconnaissable	P2
B7	En tant que fondateur, je veux transférer la propriété de mon groupe à un administrateur, afin que le groupe survive à mon départ	P2
B8	En tant que fondateur, je veux supprimer définitivement mon groupe et toutes ses données, afin d'exercer un droit d'effacement réel sur les données de ma communauté	**MVP**

> **B8 ne peut pas être différée.** Outre son fondement légal, c'est la seule story qui **exerce réellement** le contrat de purge imposé aux modules natifs (D11) et la suppression des classeurs des modules externes (M13). Sans elle, ces garanties seraient déclarées, compilées, mais jamais exécutées — donc jamais vérifiées. Une garantie qu'on n'exerce pas est une garantie qu'on ne possède pas.

Épique C — Membres, rôles et permissions

Réf.	User story	Prio
C1	En tant qu'administrateur, je veux consulter la liste des membres de mon groupe et leur rôle, afin de savoir qui fait quoi	**MVP**
C2	En tant qu'administrateur, je veux attribuer un rôle à un membre (administrateur, contributeur, lecteur), afin de déléguer les droits appropriés	**MVP**
C3	En tant qu'administrateur, je veux exclure un membre, afin de gérer les problèmes de comportement	**MVP**
C4	En tant que contributeur, je veux créer et modifier le contenu des modules activés — tier lists, fiches de composition, spotchecks, plateaux enregistrés — afin de faire vivre la base de connaissance du groupe	**MVP**
C5	En tant que contributeur, je veux supprimer un contenu devenu obsolète, afin de maintenir la base à jour	**MVP**
C6	En tant que contributeur, je veux que mes contributions soient attribuées à mon nom, afin que le travail de chacun soit identifiable	**MVP**
C7	En tant que fondateur, je veux que le rôle de contributeur ne donne aucun droit sur les membres — ni consultation de leurs données, ni attribution de rôles, ni exclusion — afin de séparer la production de contenu de l'administration de la communauté	**MVP**
C8	En tant que fondateur, je veux que le rôle de contributeur ne permette ni d'activer, ni de désactiver, ni de configurer un module, afin que la composition de l'outil reste une décision d'administration	**MVP**
C9	En tant que lecteur, je veux consulter le contenu de mon groupe sans pouvoir le modifier, afin de bénéficier du travail collectif	**MVP**
C10	En tant que membre, je veux ne jamais voir les données d'un autre groupe, afin que la confidentialité de chaque communauté soit garantie	**MVP**
C11	En tant qu'administrateur, je veux consulter un journal des actions d'administration, afin de tracer les décisions sensibles	P3

Matrice des droits

	Fondateur	Administrateur	Contributeur	Lecteur
Consulter le contenu des modules	✔	✔	✔	✔

	Fondateur	Administrateur	Contributeur	Lecteur
Créer, modifier, supprimer du contenu	✔	✔	✔	—
Inviter, exclure un membre	✔	✔	—	—
Attribuer un rôle	✔	✔	—	—
Consulter les données des membres	✔	✔	—	—
Activer, désactiver, configurer un module	✔	✔	—	—
Modifier l'identité du groupe	✔	✔	—	—
Transférer la propriété du groupe	✔	—	—	—
Supprimer le groupe et purger ses données	✔	—	—	—

> **C7 et C8 sont formulées négativement à dessein.** Ce sont des exigences de cloisonnement : elles décrivent ce que le rôle de contributeur ne doit jamais permettre, et se vérifient par des tests d'autorisation sur l'ensemble des points d'accès — pas par un écran.
>
> **C10 est une exigence transverse.** Elle ne se traduit pas par un écran mais par une propriété du système, vérifiée par des tests automatisés sur l'ensemble des points d'accès. Pour les modules externes, elle est en plus garantie par construction : un module ne reçoit jamais que les données de son propre classeur pour le groupe en cours (§3.3).

Épique D — Système de modules

Réf.	User story	Prio
D1	En tant qu'administrateur, je veux consulter le catalogue des modules disponibles avec leur description, afin de choisir ce dont mon groupe a besoin	**MVP**
D2	En tant qu'administrateur, je veux activer ou désactiver un module pour mon groupe, afin d'adapter la plateforme à nos usages	**MVP**
D3	En tant que membre, je veux ne voir dans la navigation que les modules actifs de mon groupe, afin de ne pas être encombré par des fonctionnalités inutilisées	**MVP**
D4	En tant qu'administrateur, je veux configurer chaque module activé depuis une page dédiée, afin de l'adapter à nos règles	**MVP**
D5	En tant qu'administrateur, je veux qu'une désactivation conserve les données du module, afin de pouvoir le réactiver sans rien perdre	P2
D6	En tant que développeur externe, je veux disposer d'une documentation complète du kit de développement et du format d'échange, afin d'écrire un module sans comprendre toute l'application	**MVP**
D7	En tant que développeur externe, je veux disposer d'un module d'exemple fonctionnel, afin de l'utiliser comme gabarit	**MVP**
D8	En tant que mainteneur, je veux que le build échoue si un module natif franchit une frontière d'architecture, afin que la relecture des contributions au noyau soit outillée et non uniquement humaine	**MVP**
D9	En tant qu'administrateur, je veux voir quelles données un module recevra et quelles commandes il ajoute avant de l'activer, afin de savoir à quoi j'expose mon groupe	P2
D10	En tant que fondateur, je veux purger définitivement les données d'un module, afin de nettoyer sans supprimer tout le groupe	P2
D11	En tant que mainteneur, je veux que tout module natif implémente obligatoirement l'anonymisation d'un utilisateur et la purge d'un groupe, vérifié à la compilation, afin qu'aucune donnée personnelle ne devienne inatteignable	**MVP**

> **D11 conditionne A4, A5 et B8.** Les données personnelles sont réparties dans tous les modules. Pour les modules natifs, la garantie vient d'un contrat imposé à la compilation : sans lui, une demande de suppression de compte ou d'effacement d'un groupe serait impossible à honorer complètement. C'est une contrainte structurelle : elle doit être posée avant d'écrire le premier module, car elle ne peut pas être rattrapée une fois le catalogue constitué. Pour les modules externes, la même garantie est obtenue **par construction** plutôt que par contrat : leurs données vivent dans un classeur séparé par groupe, et ils ne connaissent les membres que par pseudonyme (§3.6).
>
> **Les épiques D et M sont le cœur du projet.** Les modules ci-dessous en sont la démonstration, pas la finalité.
>
> **Pourquoi deux modules natifs engagés et non un seul, ni quatre ?** Un module unique ne démontre pas une architecture modulaire — il démontre une séparation. Avec deux, on peut vérifier qu'un module s'ajoute sans toucher à l'autre, que le jeu de commandes du bot varie réellement d'un groupe à l'autre selon les activations, et que le contrat de l'épique D s'applique à un ensemble non trivial. Au-delà, chaque module supplémentaire n'apporte plus de preuve architecturale mais consomme le temps destiné à la marketplace, qui est l'objet même du travail. Team Builder est retenu pour sa richesse technique et sa valeur de démonstration visuelle, Tier Lists parce que c'est le module le moins coûteux du catalogue et qu'il suffit à établir la pluralité. Le module externe Tournoi (M16) complète la démonstration du côté de la marketplace.

Épique M — Marketplace et développeurs externes

Nommée M pour conserver les références des épiques existantes ; elle prolonge directement l'épique D.

Réf.	User story	Prio
M1	En tant que développeur externe, je veux écrire un module en JavaScript avec le kit de développement fourni et le compiler en WebAssembly par une seule commande, afin de ne pas devoir apprendre une autre technologie	**MVP**
M2	En tant que développeur externe, je veux tester mon module sur mon poste en lui passant un paquet d'exemple, afin de développer sans installer la plateforme	P2
M3	En tant que développeur externe, je veux publier une nouvelle version de mon module, afin qu'elle soit disponible immédiatement, sans relecture ni redéploiement de la plateforme	**MVP**
M4	En tant que développeur externe, je veux que mon module soit compilé depuis mon dépôt public par une chaîne automatisée qui atteste son origine, afin que chacun puisse vérifier que le code exécuté est bien celui publié	P2
M5	En tant qu'administrateur, je veux voir le niveau de confiance d'un module (non vérifié, vérifié, certifié) et un avertissement clair pour un module non vérifié, afin de décider en connaissance de cause avant de l'activer	**MVP**
M6	En tant qu'administrateur, je veux rester sur la version installée tant que je n'ai pas accepté une mise à jour qui demande davantage de données, afin de garder le contrôle sur ce que reçoit le module	P2
M7	En tant que mainteneur, je veux désactiver un module sur toute la plateforme en une action, afin de réagir immédiatement à un module malveillant	**MVP**
M8	En tant que mainteneur, je veux qu'un module qui dépasse ses limites de calcul, de mémoire, de stockage ou de taille de réponse soit interrompu sans effet sur les autres modules ni sur les autres groupes, afin qu'un module défaillant ne dégrade pas le service	**MVP**
M9	En tant que mainteneur, je veux qu'un module en échec répété soit désactivé automatiquement et son auteur prévenu, afin de ne pas devoir surveiller le catalogue en permanence	P3
M10	En tant que mainteneur, je veux qu'une suite de modules hostiles rejoue les attaques connues à chaque changement, le build échouant si l'une d'elles réussit, afin que l'isolation soit vérifiée en continu	**MVP**
M11	En tant que développeur externe, je veux faire vérifier mon identité pour obtenir le niveau vérifié, afin que les groupes fassent davantage confiance à mon module	P2
M12	En tant que membre, je veux que les modules externes ne connaissent jamais mon identité réelle, seulement un pseudonyme propre à chaque module, afin qu'un module ne puisse pas me suivre d'un module à l'autre	**MVP**

Réf.	User story	Prio
M13	En tant que membre, je veux que les données d'un module externe soient rangées dans un classeur séparé pour chaque groupe, choisi par la plateforme et jamais par le module, afin qu'aucun module ne puisse mélanger ou lire les données de deux groupes	**MVP**
M14	En tant que développeur externe, je veux déclarer des commandes Discord pour mon module, afin que le bot les relaie sur les serveurs des groupes qui l'ont activé	P3
M15	En tant que développeur vérifié, je veux afficher une interface libre dans un cadre isolé quand les composants de la plateforme ne suffisent pas, afin de proposer des écrans plus riches	P3
M16	En tant que mainteneur, je veux disposer d'un module de démonstration Tournoi, écrit dans un dépôt séparé comme le ferait un développeur externe, afin de prouver la chaîne de bout en bout	**MVP**

> **M8, M10, M12 et M13 sont des propriétés du système, comme C10.** Elles ne se traduisent pas par un écran : elles se vérifient par la suite de modules hostiles (M10), où chaque tentative — lire un autre classeur, dépasser un plafond, renvoyer une réponse malformée — doit échouer.
>
> **M4 est classée P2 sans affaiblir la sécurité.** Sans elle, la publication passe par le dépôt manuel du fichier compilé : le bac à sable protège exactement de la même façon. Seule la traçabilité de l'origine du code est différée.

Épique E — Module *Team Builder* (engagé)

Réf.	User story	Prio
E1	En tant que membre, je veux placer des personnages sur une grille hexagonale par glisser-déposer, afin de construire une situation de jeu	**MVP**
E2	En tant que membre, je veux voir en temps réel les synergies activées par ma composition, afin de valider sa cohérence	**MVP**
E3	En tant que membre, je veux attribuer un niveau d'étoiles et des objets à chaque unité, afin de représenter fidèlement une situation	**MVP**
E4	En tant que membre, je veux partager un plateau par simple lien, afin d'en discuter sans rien enregistrer	**MVP**
E5	En tant que membre, je veux qu'un lien reçu reconstruise exactement le plateau d'origine, afin que le partage soit fiable	**MVP**
E6	En tant que membre, je veux afficher deux plateaux en vis-à-vis, afin de raisonner sur une confrontation	**MVP**
E7	En tant que membre, je veux utiliser l'éditeur au doigt sur mobile ou tablette, afin de travailler ailleurs que devant un ordinateur	P2
E8	En tant que membre, je veux enregistrer un plateau dans la bibliothèque de mon groupe avec un titre, afin de le retrouver plus tard	P2
E9	En tant que membre, je veux annoter un plateau enregistré, afin d'expliquer un choix de placement	P3

Épique F — Module *Compos* (différé)

Réf.	User story	Prio
F1	En tant que contributeur, je veux créer une fiche de composition à partir d'un plateau de référence, afin de documenter une stratégie	P2
F2	En tant que contributeur, je veux décrire les objets recommandés et les conseils de jeu d'une composition, afin de la rendre exploitable	P2
F3	En tant que membre, je veux consulter les fiches de composition de mon groupe, afin d'apprendre ce que la communauté a documenté	P2
F4	En tant que contributeur, je veux déclarer les conditions de viabilité d'une composition, afin d'indiquer quand elle est jouable ou non	P2

Réf.	User story	Prio
F5	En tant que contributeur, je veux documenter les ouvertures de partie recommandées, afin de guider les premiers tours	P3
F6	En tant que membre, je veux voir l'historique des modifications d'une fiche, afin de suivre son évolution au fil des patchs	P3
F7	En tant que membre, je veux commenter une fiche, afin d'engager la discussion sans passer par Discord	P3

Épique G — Module *Tier Lists* (engagé)

Réf.	User story	Prio
G1	En tant que contributeur, je veux classer des éléments par tiers en les faisant glisser, afin d'exprimer le consensus du groupe	**MVP**
G2	En tant que groupe, je veux disposer de cinq tier lists distinctes (compositions, augments, objets composables, emblèmes, objets forgés), afin de couvrir les décisions du jeu	**MVP**
G3	En tant que membre, je veux consulter les tier lists de mon groupe, afin de savoir ce qui est considéré comme fort en ce moment	**MVP**
G4	En tant que groupe, je veux que nos classements soient les nôtres et ne soient pas visibles des autres groupes, afin que notre analyse reste interne	**MVP**
G5	En tant que contributeur, je veux justifier un classement par une note courte, afin d'expliquer un choix contre-intuitif	P2

Épique H — Module *Spotchecks* (différé)

Réf.	User story	Prio
H1	En tant que contributeur, je veux créer un spotcheck présentant une situation et plusieurs choix possibles, afin de soumettre une décision au débat	P2
H2	En tant que membre, je veux consulter les spotchecks de mon groupe, afin de m'entraîner à la prise de décision	P2
H3	En tant que membre, je veux donner mon avis sur un spotcheck, afin de participer au débat	P3
H4	En tant que contributeur, je veux publier la réponse argumentée après un délai, afin que la discussion ait lieu avant la correction	P3

Épique I — Modules bonus

Ces modules ne sont pas engagés. Ils sont mentionnés parce qu'ils constituent la démonstration d'usage de la marketplace : ce sont eux qui prouveront que la plateforme est réellement extensible. Ils ont vocation à être écrits comme modules externes.

Module envisagé	Objet
Entraînement	Exercices de positionnement, seul ou entre deux membres
Tournois	Organisation d'événements internes au groupe — une version minimale est réalisée comme module de démonstration (M16)
Progression	Suivi de l'activité individuelle dans le temps
Calendrier	Sessions d'entraînement et disponibilités
Fil d'activité	Journal des contributions du groupe

2.3 Bot Discord

Le bot est développé indépendamment de l'application web et consomme son API. Aucune de ses fonctionnalités n'est classée MVP.

Épique J — Consultation depuis Discord

Réf.	User story	Prio
J1	En tant que membre, je veux afficher un plateau dans un salon à partir de son lien, afin d'en discuter sans quitter Discord	P2
J2	En tant que membre, je veux afficher une tier list de mon groupe par une commande, afin de vérifier un classement en pleine discussion	P2
J3	En tant que membre, je veux consulter le tiers attribué à un ou plusieurs augments, afin de décider rapidement en cours de partie	P2
J4	En tant que membre, je veux consulter le tiers attribué à un ou plusieurs objets, afin de choisir un équipement	P2
J5	En tant que membre, je veux bénéficier de l'autocomplétion sur les noms d'augments et d'objets, afin de ne pas avoir à les orthographier exactement	P2
J6	En tant que membre, je veux afficher un spotcheck dans un salon, afin de lancer le débat auprès de ceux qui ne sont pas sur le site	P3

Épique K — Publication et synchronisation

Réf.	User story	Prio
K1	En tant que contributeur, je veux publier les fiches de composition de mon groupe dans Discord, afin qu'elles soient consultables là où vit la communauté	P2
K2	En tant que contributeur, je veux que le bot crée et organise lui-même les salons nécessaires, afin de ne pas le faire à la main	P2
K3	En tant que contributeur, je veux relancer une publication sans créer de doublon, afin de pouvoir corriger une fiche	P2
K4	En tant que membre, je veux que les discussions humaines d'un salon soient préservées lors d'une republication, afin de ne pas perdre nos échanges	P2
K5	En tant que membre, je veux exporter un plateau depuis le site vers un salon Discord, afin de le soumettre au groupe	P3

Épique L — Administration du bot

Réf.	User story	Prio
L1	En tant qu'administrateur, je veux lier mon serveur Discord à mon groupe depuis le site, afin d'activer le bot pour ma communauté	P2
L2	En tant qu'administrateur, je veux que le bot fonctionne sur le serveur de chaque groupe indépendamment, afin que plusieurs communautés puissent l'utiliser	P2
L3	En tant qu'administrateur, je veux configurer les salons utilisés par le bot, afin qu'il publie au bon endroit	P2
L4	En tant qu'administrateur, je veux que les commandes disponibles sur mon serveur correspondent aux modules activés par mon groupe, afin de ne pas proposer des commandes sans effet	P2
L5	En tant qu'administrateur, je veux que les commandes d'administration soient réservées aux rôles autorisés, afin qu'un membre ne puisse pas déclencher une publication massive	P2

> **L4 est la user story la plus structurante du bot** : elle impose que les modules — natifs comme externes — déclarent leurs commandes dans leur manifeste, et que le jeu de commandes enregistré varie d'un serveur à l'autre. C'est la preuve que la modularité traverse toute la plateforme, pas seulement le web.

---

3. La marketplace de modules

3.1 Le problème

Un module externe est du code écrit par un inconnu, qui doit s'exécuter sur l'instance partagée par toutes les communautés. Malveillant ou simplement bogué, il ne doit pouvoir abîmer que son propre périmètre : **jamais lire les données d'un autre groupe, ni les secrets de la plateforme, ni ralentir le service pour les autres**.

La plateforme .NET n'offre plus aucun moyen d'isoler du code tiers au sein d'un même processus : le mécanisme qui le permettait (*Code Access Security*) a été abandonné. Charger un module externe comme une simple bibliothèque lui donnerait les mêmes droits que la plateforme elle-même, et donc l'accès aux données de tous les groupes.

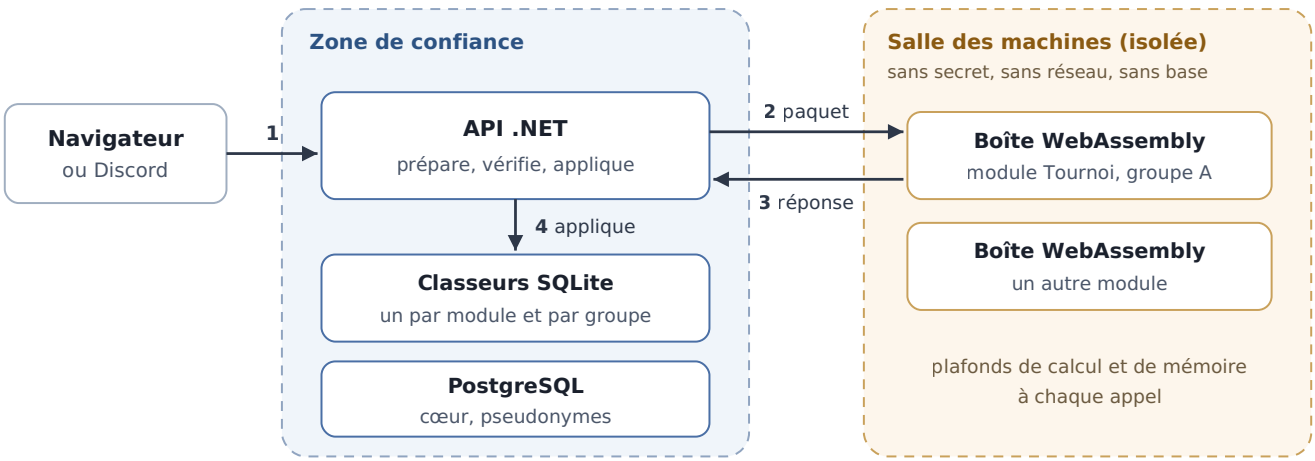
3.2 Exigences

Exigence	Détail
Vrai code	Une logique serveur, une interface et des données propres — pas une simple configuration
Sécurité maximale	Aucune lecture possible des données d'un autre groupe ni des secrets de la plateforme
Pas de relecture	Le mainteneur ne relit pas le code de chaque version
Disponibilité immédiate	Une version publiée est utilisable sans redéployer la plateforme
Autonomie du développeur	Le module vit et se maintient dans le dépôt de son auteur
Instance unique	Ni base de données ni serveur par groupe ou par module
Budget	Hébergement total inférieur à 10 € par mois

3.3 Le principe : une boîte fermée qui reçoit un paquet et rend une réponse

Un module externe est compilé en **WebAssembly** et s'exécute dans un **bac à sable**. Par construction, il n'a accès à rien : ni fichiers, ni réseau, ni base de données, ni mémoire de la plateforme. Il ne peut que calculer.

Surtout, **il ne demande rien pendant son exécution**. La plateforme lui donne tout ce dont il a besoin dès le départ ; il rend une réponse ; la plateforme décide de ce qu'elle en fait.



Étape	Où	Ce qui se passe
1. Événement	Navigateur ou Discord	Un membre clique sur un bouton du module, ou tape une de ses commandes
2. Préparation	API, zone de confiance	La plateforme ouvre le classeur du module pour ce groupe et prépare un **paquet** : l'événement, le contenu du classeur, le contexte (pseudonyme et rôle du membre, données de jeu utiles)
3. Exécution	Salle des machines, zone isolée	Le module tourne dans sa boîte, avec des plafonds de calcul et de mémoire, et rend une **réponse** : les modifications à apporter à son classeur, la description de l'écran à afficher, les messages éventuels
4. Vérification et application	API, zone de confiance	La réponse est validée (format, taille, portée limitée au classeur du module pour ce groupe), puis appliquée et affichée

Les appels d'un même module pour un même groupe sont traités **l'un après l'autre**, ce qui exclut toute écriture concurrente dans un classeur. Pour le développeur, rien de ce mécanisme n'est visible : le kit de développement lui fournit des fonctions comme `enregistrer()`, `supprimer()` ou `afficher()`, qui alimentent simplement la réponse.

> **Pourquoi ne pas laisser le module interroger la plateforme pendant qu'il tourne ?** Chaque question possible serait une porte supplémentaire à surveiller. En lui donnant tout au départ, la seule chose à contrôler est sa réponse. C'est le modèle des **Shopify Functions**, qui exécutent du code de partenaires externes sur l'infrastructure de Shopify : des données en entrée, une liste d'opérations en sortie, appliquées par la plateforme.

3.4 Les couches de protection

Les protections sont **indépendantes** : la défaillance de l'une ne suffit pas à compromettre le système.

Couche	Ce qu'elle garantit
1. Bac à sable WebAssembly	Le module n'a accès qu'à son paquet. Une boucle infinie est interrompue par le plafond de calcul, une consommation excessive par le plafond de mémoire. Moteur Wasmtime, en version à support long
2. Salle des machines séparée	Les boîtes tournent dans un conteneur distinct de l'API : sans secret, sans accès réseau sortant, sans accès aux bases, système de fichiers en lecture seule, processeur et mémoire plafonnés. Un module qui parviendrait à sortir de sa boîte n'y trouverait rien d'exploitable
3. Réponse vérifiée	Chaque réponse est validée par la plateforme. Un module ne peut modifier que son propre classeur, pour le groupe en cours, dans les limites de taille fixées
4. Classeur par module et par groupe	Un fichier SQLite séparé, choisi par la plateforme et jamais par le module, plafonné en taille. Le module n'écrit jamais de SQL : la plateforme lit et écrit pour lui
5. Pseudonymes	Le module ne connaît les membres que par un pseudonyme propre à lui. La correspondance reste dans PostgreSQL, hors de sa portée
6. Interface déclarative	Le module décrit son écran avec les composants de la plateforme ; aucun code tiers ne s'exécute dans le navigateur des membres
7. Niveaux de confiance	Non vérifié (avec avertissement), vérifié, certifié. Arrêt d'urgence global en une action
8. Modules hostiles	Une suite de modules malveillants, exécutée en intégration continue, tente chaque attaque connue ; le build échoue si l'une réussit

3.5 Cycle de vie d'un module

1. **Écriture.** Le développeur écrit son module en JavaScript dans son propre dépôt, avec le kit de développement TGFT (types, fonctions de réponse, composants d'interface).

2. **Compilation.** L'outil **Javy** transforme le JavaScript en un fichier WebAssembly. Dans le MVP, le développeur compile lui-même et dépose le fichier avec son manifeste. En P2 (M4), la compilation est faite par GitHub Actions depuis le dépôt public, et la plateforme vérifie l'attestation qui prouve l'origine du fichier.
3. **Publication.** La plateforme valide le manifeste (nom, version, écrans, réglages, commandes, taille de classeur demandée) et le fichier, puis ajoute la version au catalogue, sans redéploiement.
4. **Activation.** L'administrateur d'un groupe voit le niveau de confiance et les données que le module recevra, puis l'active pour son groupe.
5. **Exécution.** À chaque appel, selon le déroulé du §3.3. Le fichier compilé est conservé en cache.
6. **Retrait.** Désactivation par le groupe avec conservation des données (D5), purge (D10), ou arrêt d'urgence sur toute la plateforme (M7).

3.6 Données personnelles

L'architecture rend les obligations RGPD simples à honorer pour les modules externes, sans rien exiger d'eux :

- **Purger un groupe (B8)** : supprimer ses classeurs. Aucune action n'est demandée au module.
- **Anonymiser un utilisateur (A5)** : supprimer la correspondance entre l'utilisateur et ses pseudonymes. Ses contributions restent en place, attachées à un pseudonyme qui ne désigne plus personne.
- **Limite** : un texte libre saisi par un membre (un nom cité dans un commentaire, par exemple) n'est pas couvert par ce mécanisme. Les conditions de publication interdisent donc aux modules de solliciter des données personnelles, et la purge d'un groupe reste, elle, complète.

3.7 Modèles de référence

Chaque brique s'appuie sur un modèle déjà exploité en production par une grande plateforme :

Brique TGFT	Modèle en production
Code tiers exécuté en WebAssembly, paquet en entrée, opérations en sortie	Shopify Functions
Compilation de JavaScript en WebAssembly	Javy, créé par Shopify pour ses Functions, aujourd'hui maintenu par la Bytecode Alliance
Une base de données séparée par installation d'un module	Atlassian Forge SQL
Interface décrite avec les composants de la plateforme, sans accès à la page	Shopify UI extensions

3.8 Limites assumées

- **Pas de lecture en cours d'exécution** : tout ce dont le module a besoin doit figurer dans le paquet.
- **Classeur plafonné** (quelques mégaoctets), puisqu'il est chargé à chaque appel. Largement suffisant pour un tournoi ou une tier list.
- **Pas d'accès réseau**, pas de temps réel, pas d'appel à un service extérieur.
- **Intégration visuelle limitée** aux composants de la plateforme, sauf le cadre isolé réservé aux développeurs vérifiés (M15, P3).
- **Aucune isolation n'est infaillible.** En avril 2026, Wasmtime a corrigé deux failles critiques permettant de sortir du bac à sable, limitées à un compilateur secondaire et aux processeurs ARM, sur toutes ses versions supportées. C'est ce qui justifie la salle des machines séparée (couche 2) et trois règles d'exploitation : un serveur x86, le compilateur par défaut, et l'application rapide des mises à jour de sécurité.

3.9 Périmètre dans le projet

Implémenté (MVP)	Conçu et documenté, implémenté si le MVP est tenu
Salle des machines Wasmtime, avec plafonds de calcul et de mémoire	Compilation depuis le dépôt avec attestation (M4)
Format d'échange paquet / réponse et kit de développement JavaScript	Vérification et certification des développeurs (M11)
Classeurs SQLite par module et par groupe, pseudonymes	Accord de l'administrateur pour les mises à jour (M6)
Interface déclarative et composants de base	Outil de test local (M2)
Publication par dépôt du fichier compilé	Désactivation automatique (M9), commandes Discord (M14)
Niveau non vérifié, avertissement, arrêt d'urgence	Cadre isolé pour les développeurs vérifiés (M15)
Suite de modules hostiles en intégration continue	
Module de démonstration Tournoi (M16)	

Les mécanismes différés sont **conçus et documentés** dans ce cahier des charges même s'ils ne sont pas implémentés : l'isolation entre groupes ne dépend d'aucun d'eux.

> **Premier jalon technique et plan de repli.** Le premier livrable de la marketplace est un prototype de bout en bout : un module JavaScript compilé avec Javy, exécuté depuis .NET avec ses plafonds. S'il échoue, l'exécution bascule sur **Jint**, un interpréteur JavaScript écrit en C#, placé dans la même salle des machines. Ses auteurs précisant qu'il ne constitue pas à lui seul une frontière de sécurité, la salle des machines isolée (couche 2) deviendrait alors la protection principale. Le reste de l'architecture — paquet, réponse, classeurs, pseudonymes, interface — reste identique.

---

4. Stack technique et justifications

4.1 Vue d'ensemble

Couche	Technologie
Back-end	ASP.NET Core (C#)
Front-end	Angular (TypeScript)
Base de données	PostgreSQL
Accès aux données	Entity Framework Core + Dapper
Temps réel	SignalR
Bot Discord	Discord.Net (C#)
Rendu d'images	ImageSharp
Cache	Cache mémoire applicatif (.NET) + cache disque pour les images
Exécution des modules externes	Wasmtime pour .NET (version à support long)
Compilation des modules externes	Javy (JavaScript vers WebAssembly)
Stockage des modules externes	SQLite (Microsoft.Data.Sqlite)
Conteneurisation	Docker / Docker Compose

Couche	Technologie
Proxy inverse	Caddy
Intégration continue	GitHub Actions
Tests	xUnit, Testcontainers, NetArchTest

4.2 Justifications

ASP.NET Core (C#) — Typage fort et écosystème mature, adaptés à un projet dont l'enjeu principal est la rigueur architecturale. Avantage décisif ici : **le bot Discord et la salle des machines peuvent être écrits dans le même langage**, ce qui permet de partager les contrats de données entre l'API, le bot et l'exécution des modules, et de les faire vérifier à la compilation. Avec un bot en Python ou en JavaScript, ces contrats seraient dupliqués manuellement et la moindre évolution de l'API casserait le bot silencieusement.

Angular — Framework structurant, adapté aux applications à nombreux écrans et formulaires. Son système de chargement différé est directement aligné sur la modularité du projet : les écrans d'un module natif désactivé ne sont pas chargés par le navigateur, donc un module inactif ne coûte rien à l'utilisateur. Pour les modules externes, Angular affiche la description d'écran renvoyée par le module avec les composants de la plateforme : **aucun code inconnu n'est chargé dans le navigateur**.

PostgreSQL — Choisi sur deux critères précis liés à l'architecture :

- Le **Row Level Security** permet d'appliquer l'isolation entre groupes au niveau du moteur de base de données. C'est un **second rempart indépendant du code applicatif** : même si une requête oubliait son filtre, la base ne renverrait pas les données d'un autre groupe.
- Le type **JSONB** indexable convient à la configuration des modules, qui est par nature à schéma variable et appelé à évoluer sans modification du noyau. L'alternative — une colonne par réglage, ou une table clé-valeur — serait rigide ou inexploitable.

Il conserve aussi la correspondance entre les utilisateurs et leurs pseudonymes auprès des modules externes, hors de portée de ceux-ci.

Entity Framework Core + Dapper — Approche mixte volontaire.

Le multi-tenant impose que chaque requête soit restreinte au groupe courant. Confier cette restriction à la vigilance du développeur, requête par requête, est la source classique des fuites de données : une seule omission suffit, et elle est silencieuse. Entity Framework Core dispose de **filtres de requête globaux**, déclarés une fois par entité et appliqués automatiquement à toutes les requêtes, y compris celles écrites plus tard. L'isolation devient structurelle plutôt que déclarative.

Dapper est conservé pour les quelques lectures complexes où le contrôle du SQL apporte un gain réel, sous réserve de passer par la couche d'accès dédiée.

SignalR — Communication temps réel intégrée nativement au back-end, avec repli automatique selon les capacités du client. Utile pour les modules natifs interactifs envisagés, et pour les retours de progression sur les traitements longs.

Discord.Net — Bibliothèque mature pour le bot. Le choix du C# plutôt que de discord.js ou discord.py est assumé : l'écosystème est plus petit, mais le partage de code avec l'API prime.

ImageSharp — Rendu d'images côté serveur, purement managé, sans dépendance native — donc conteneur léger. Permet de générer les images de plateau exploitables à la fois par le site et par le bot.

Cache mémoire applicatif + cache disque — Aucun service de cache externe n'est utilisé : les besoins sont couverts par des moyens déjà disponibles.

Les données de jeu ne changent qu'à chaque nouvelle version publiée par Riot. Elles sont donc chargées une fois et conservées en mémoire par l'application, puis rechargées uniquement lorsque la vérification quotidienne détecte un changement. Les images de plateau sont coûteuses à générer et souvent redemandées — un lien partagé dans un

salon est ouvert par plusieurs personnes. Chaque image est donc générée une fois puis conservée sur disque, sous un nom dérivé du contenu du plateau et de la version de jeu, de sorte qu'un plateau modifié produit automatiquement un nouveau fichier. Le même cache disque conserve les modules externes une fois compilés.

Un cache externe type Redis n'apporterait quelque chose qu'en cas de déploiement sur plusieurs instances, ce qui n'est pas le modèle retenu. Il est écarté pour ne pas ajouter un service à exploiter sans bénéfice.

Wasmtime pour .NET — Le moteur WebAssembly de la Bytecode Alliance, conçu pour exécuter du code non fiable, avec un processus public de publication des failles et de leurs correctifs. Sa bibliothèque .NET officielle permet d'écrire la salle des machines en C#, sans introduire de nouveau langage dans la pile. La version 48 est une **version à support long** : ses correctifs de sécurité sont garantis pendant 24 mois. Les plafonds de calcul et de mémoire sont fixés à chaque appel.

Javy — Transforme un module JavaScript en WebAssembly. Créé par Shopify pour ses Functions, il est aujourd'hui maintenu par la Bytecode Alliance. Le choix de JavaScript abaisse la barrière d'entrée pour les développeurs de la communauté. Le moteur JavaScript est lui-même compilé dans le module : un défaut de ce moteur reste donc confiné dans la boîte.

SQLite — Un fichier par module et par groupe : séparation physique des données, purge par simple suppression de fichier, quota par taille de fichier, et **aucun serveur supplémentaire à exploiter**. Le module n'écrit jamais de SQL : la plateforme lit et écrit pour lui.

Docker / Docker Compose — Reproductibilité de l'environnement et démarrage de l'ensemble de la pile par une commande unique, condition d'une démonstration fiable. La salle des machines tourne dans **son propre conteneur**, sans secret, sans accès réseau sortant, en lecture seule, avec processeur et mémoire plafonnés.

Caddy — Le point d'entrée public : il reçoit toutes les requêtes venant d'internet et les distribue vers le bon service interne, l'API n'étant ainsi jamais exposée directement.

Il gère seul les **certificats HTTPS**, leur obtention comme leur renouvellement, là où une configuration manuelle est une source classique de pannes. Il sert directement les fichiers de l'interface Angular une fois compilée, ainsi que les images déjà générées, comme de simples fichiers statiques : ces requêtes n'atteignent jamais l'application. Cela évite d'exploiter un second serveur web dédié à l'interface. La salle des machines n'est jamais exposée : seule l'API peut l'appeler.

GitHub Actions — Exécution automatique à chaque contribution de la compilation, des tests et des contrôles d'architecture. À terme (M4), il compile aussi les modules externes depuis leur dépôt public et produit l'attestation de leur origine.

xUnit, Testcontainers, NetArchTest — Trois outils pour trois besoins :

- **xUnit** pour les tests unitaires du domaine ;
- **Testcontainers** pour lancer une vraie base de données jetable pendant les tests d'intégration, condition nécessaire pour tester réellement l'isolation entre groupes, ainsi que la vraie salle des machines face à la suite de modules hostiles ;
- **NetArchTest** pour exprimer les règles d'architecture sous forme de tests — un module natif qui référencerait directement la base du noyau ou un autre module fait échouer le build.

---

5. Défis techniques et points d'intérêt

1. Isolation multi-tenant sans frontière physique. Garantir qu'aucun groupe n'accède jamais aux données d'un autre, sur une instance unique partagée, sans conteneur dédié par groupe. La garantie repose sur l'empilement de mécanismes indépendants — filtres globaux, Row Level Security, classeurs séparés pour les modules externes, tests automatisés — plutôt que sur une séparation matérielle.

2. Exécuter du code inconnu sans le relire. Faire tourner, sur l'instance partagée, du code dont personne n'a vérifié le contenu. La plateforme .NET n'offrant aucune isolation au sein d'un même processus, la garantie est

produite autrement : un bac à sable WebAssembly, une salle des machines séparée sans rien d'exploitable, et un modèle d'échange où le module ne peut rien demander, seulement répondre.

3. Concevoir un format d'échange à double usage. Assez riche pour qu'un développeur inconnu construise un module réellement utile, comme un tournoi ; assez étroit pour que chaque réponse reste entièrement vérifiable. Le partage des mêmes contrats entre modules natifs et externes est le garde-fou contre une extensibilité purement décorative.

4. Modularité sur un framework front compilé. Angular ne charge pas de code inconnu à l'exécution. Les modules natifs reposent sur un registre généré au build et un chargement différé, pour un coût nul quand ils sont inactifs ; les modules externes, sur une interface déclarative rendue avec les composants de la plateforme. Concilier les deux dans une navigation unique, contrôlée par groupe, est un problème d'architecture à part entière.

5. Éditeur hexagonal tactile avec état encodé en URL. Glisser-déposer fiable sur grille hexagonale, au doigt comme à la souris, et encodage complet de la situation dans une URL compacte — contrainte forte, puisque les identifiants du jeu changent tous les quatre mois et qu'un lien ancien doit rester interprétable.

6. Bot multi-serveurs à commandes variables. Un bot unique desservant plusieurs communautés, dont le jeu de commandes enregistré diffère sur chaque serveur selon les modules — natifs comme externes — activés par le groupe correspondant, le tout en respectant les quotas imposés par Discord.

7. Les tests comme mécanisme de sécurité. Les tests d'isolation, d'architecture et la suite de modules hostiles ne vérifient pas la protection : **ils sont la protection**. Chaque attaque connue est rejouée à chaque changement. Les supprimer ne réduirait pas la confiance dans le système, cela supprimerait la garantie elle-même. Ils sont donc bloquants en intégration continue au même titre que la compilation.

8. Une publication sans relecture, mais traçable. Garantir que le code exécuté est exactement celui publié, compilé depuis sa source sur une infrastructure jetable et attesté, sans que la compilation n'expose jamais le serveur de la plateforme (M4).